

Member ID: _____

Time: _____

Rank: _____



C++ Programming

(335)

National 2025

PRODUCTION:

National_CPlusPlus

_____ (530 points)

Test Time: 90 minutes

OVERVIEW

In this challenge, you will develop a C++ console application called "**Bowling Score Calculator**" that computes the final score of one or more bowling games. Your program will read ball scores from a file named **Bowling.txt** and calculate the final score using standard bowling rules.

How Bowling Is Calculated

A standard bowling game consists of **10 frames**. In each frame, the bowler has up to **two balls** to knock down 10 pins. Scoring rules include:

- **Strike:**
 - When the bowler knocks down all 10 pins with the first ball of a frame.
 - Score for that frame = 10 plus the sum of the next **two** ball scores.
- **Spare:**
 - When the bowler knocks down all 10 pins in a frame using two balls.
 - Score for that frame = 10 plus the score of the **next** ball.
- **Open Frame:**
 - When fewer than 10 pins are knocked down in the frame.
 - Score = sum of the two ball scores in that frame.
- **10th Frame Special Rules:**
 - If a strike is rolled in the 10th frame, two extra balls are provided.
 - If a spare is rolled in the 10th frame, one extra ball is provided.
 - Extra balls are used only for the bonus calculation of the 10th frame.

The final score is the sum of the scores for all 10 frames.

SUBMISSION INSTRUCTIONS

- Verify that your project compiles and runs without errors.
- Ensure that your output matches the specified format exactly.
- Include all required files (source code and **Bowling.txt**) in your project submission.
- Zip your project folder, named with your contestant number, and uploaded to <https://upload.bpa.org/>.
- Your solution must be fully automated; no additional user input should be required once the program starts.

FILE STRUCTURE: Bowling.txt

- **Multiple Games:**
 - **Bowling.txt** may contain multiple bowling games.
 - Each game is represented on a separate line.
- **Game Format:**
 - Each line is a sequence of integers representing the number of pins knocked down on each ball.
 - The integers are **comma-separated** (whitespace around commas is optional).
 - The number of integers per game can vary (typically between 12 and 21) depending on strikes, spares, and extra balls in the 10th frame.

- **Example Line (Perfect Game):**

```
10,10,10,10,10,10,10,10,10,10,10,10
```

- **Example Line (Mixed Game):**

```
7,3,8,1,10,7,2,6,4,10,8,1,8,1,9,0,10,7,3,8
```

Each line represents a complete game. Your program should parse each line separately, compute the final score, and output the result.

CLASS AND STRUCTURE REQUIREMENTS

Your solution must include the following classes and adhere to the specified naming conventions and structure:

1. **BowlingGame Class**
 - **Purpose:** Represents a single bowling game.
 - **Private Data Member:**
 - A container (e.g., `std::vector<int>`) to store the sequence of ball scores for the game.
 - **Constructor:**
 - Accepts a container (or other appropriate parameters) of integers representing ball scores.
 - **Public Method:**
 - `int calculateScore():` Computes and returns the final score for the game using standard bowling rules.
2. **(Optional) Additional Classes or Structures**
 - You may create additional classes or helper functions (for example, a `Frame` class) to assist with the score calculation.
 - All additional classes must follow proper object-oriented design principles.
3. **Main Program Structure**
 - **File Name:** Your main source file must be named **BowlingScoreCalculator.cpp**.
 - **Responsibilities:**
 - Open and read **Bowling.txt**.

- Parse each line into a sequence of ball scores.
 - Instantiate a `BowlingGame` object for each game.
 - Call the `calculateScore()` method to compute the final score.
 - Print a header and output the final score for each game.
 - **Error Handling:**
 - Display an appropriate error message if **Bowling.txt** cannot be opened.
 - Robustly handle parsing errors or invalid data.
4. **Coding Standards**
- Use object-oriented programming principles; no global variables are allowed.
 - Follow a consistent naming convention.
 - Include appropriate comments to explain code logic where necessary.
-

OUTPUT REQUIREMENTS

- **Header:**

The program must print the following header:

```
Bowling Score Calculator
```
 - **Game Scores:**

For each game in **Bowling.txt**, print the final score in the following format:

```
Game 1 Final Score: <score>
Game 2 Final Score: <score>
...
```
 - **Detailed Breakdown (Optional):**

You may include a frame-by-frame breakdown of scores if desired, but the final score must be clearly printed.
 - **Format:**

The output must match the specified format exactly.
-

DETAILED RUBRIC

Category	Sub-Category	Description	Points	Points Awarded
Solution and Project Setup	Folder Structure & Naming	Correct folder creation using your contestant number.	5	
	Complete Submission	Entire project (all required source files) is submitted and accessible.	5	
Program Execution	Compilation and Run	Program compiles without errors and runs automatically.	20	
	Header Output	Header "Bowling Score Calculator" is printed exactly.	10	
	Final Score Calculation	Final score for each game is correctly computed.	20	
	Output Format	Output (header and game scores) matches the specification exactly.	20	
	Detailed Breakdown (Optional)	If provided, the frame-by-frame breakdown is accurate and clearly formatted.	30	
File Handling and Data Parsing	File Opening and Error Handling	Bowling.txt is opened correctly; an appropriate error message is displayed on failure.	30	
	Parsing of Input Data	Correct parsing of comma-separated integer values from each line.	40	
	Data Conversion	Accurate conversion of string data to integers.	30	
	Order Preservation	Input data for each game is processed in the correct order.	40	
	File Closure and Robustness	File is properly closed; code robustly handles potential parsing errors.	10	
	Invalid Data Handling	Adequate handling of invalid or out-of-range data.	30	
Bowling Score Calculation Algorithm	Open Frames, Spares, and Strikes	Accurate computation of scores for standard frames, including bonus calculations.	80	
	Handling the 10th Frame	Correct implementation of extra ball logic in the 10th frame.	60	
	Algorithm Efficiency and Accuracy	The scoring algorithm is efficient and accurate under all scenarios.	40	

Category	Sub-Category	Description	Points	Points Awarded
	Object-Oriented Design	Proper encapsulation of game logic within classes and methods (no global variables).	40	
	Code Readability and Documentation	Consistent naming, clear structure, and sufficient commenting throughout the code.	20	
Total			530	

// Contestant Number: _____

// Bowling Score Calculator

// File: BowlingScoreCalculator.cpp

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cstdlib>
using namespace std;
```

// Class representing a single bowling game.

```
class BowlingGame {
```

```
private:
```

```
    vector<int> balls; // All ball scores for the game
```

```
public:
```

```
    BowlingGame(const vector<int>& scores) : balls(scores) {}
```

// Calculate final score using standard bowling rules.

```
int calculateScore() {
```

```
    int score = 0;
```

```
    int index = 0; // Index into the balls vector
```

```
    // A standard game has 10 frames.
```

```
    for (int frame = 0; frame < 10; frame++) {
```

```
        // Strike: first ball is 10.
```

```
        if (balls[index] == 10) {
```

```
            score += 10 + balls[index + 1] + balls[index + 2];
```

```
            index += 1; // Strike uses one ball in the frame.
```

```
        }
```

```
        // Spare: two balls sum to 10.
```

```
        else if (balls[index] + balls[index + 1] == 10) {
```

```
            score += 10 + balls[index + 2];
```

```
            index += 2;
```

```
        }
```

```
    } // Open frame.
```

```
        else {
            score += balls[index] + balls[index + 1];
            index += 2;
        }
    }
    return score;
}
};

int main() {
    ifstream infile("Bowling.txt");
    if (!infile) {
        cout << "Error: Could not open Bowling.txt" << endl;
        return 1;
    }

    cout << "Bowling Score Calculator" << endl << endl;

    string line;
    int gameNumber = 1;
    // Process each game (each line in the file).
    while (getline(infile, line)) {
        if (line.empty()) continue; // Skip empty lines

        vector<int> scores;
        stringstream ss(line);
        string token;
        // Split the line by commas.
        while (getline(ss, token, ',')) {
            // Trim leading and trailing whitespace.
            size_t start = token.find_first_not_of(" \t");
            size_t end = token.find_last_not_of(" \t");
            if (start != string::npos && end != string::npos)
                token = token.substr(start, end - start + 1);
            // Convert token to integer.
            int val = stoi(token);
            scores.push_back(val);
        }

        BowlingGame game(scores);
        int finalScore = game.calculateScore();
        cout << "Game " << gameNumber << " Final Score: " << finalScore << endl;
        gameNumber++;
    }

    infile.close();
    return 0;
}
```